TEAM GUIDE | PROJECT EXPLAINED SIMPLY

Docker HEALTHCHECK for a Spring Boot Application

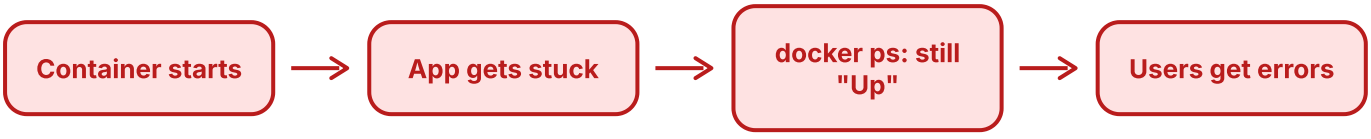
Spring Boot > Actuator > Docker HEALTHCHECK > PostgreSQL > Docker Compose

1. What is our project?

Our project **teaches Docker how to check whether an application is really working**, not just whether its container is running.

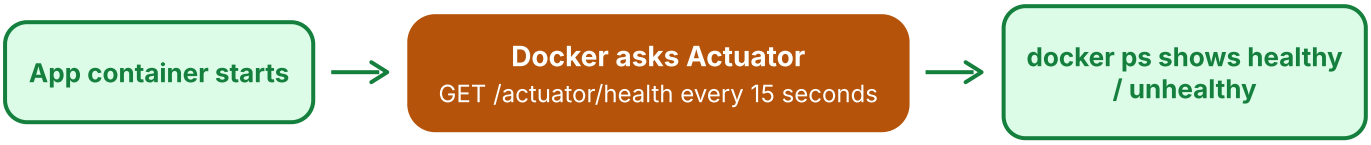
A container can show **Up** while the application inside is frozen, broken or cut off from its database. We add a Docker **HEALTHCHECK** that asks Spring Boot for its health status, so **docker ps** can tell us the truth.

WITHOUT a health check (we only see "Up")



Docker only knows the process exists. Nobody notices the problem until users complain.

WITH our HEALTHCHECK (Docker checks for us)



Problems become visible in under a minute, with no manual checking.

2. Why did we build this project?

A running container does not prove a working application. Our project helps us:

Spot hidden failures
A running container can still hold a broken app.

See status at a glance
docker ps shows healthy or unhealthy.

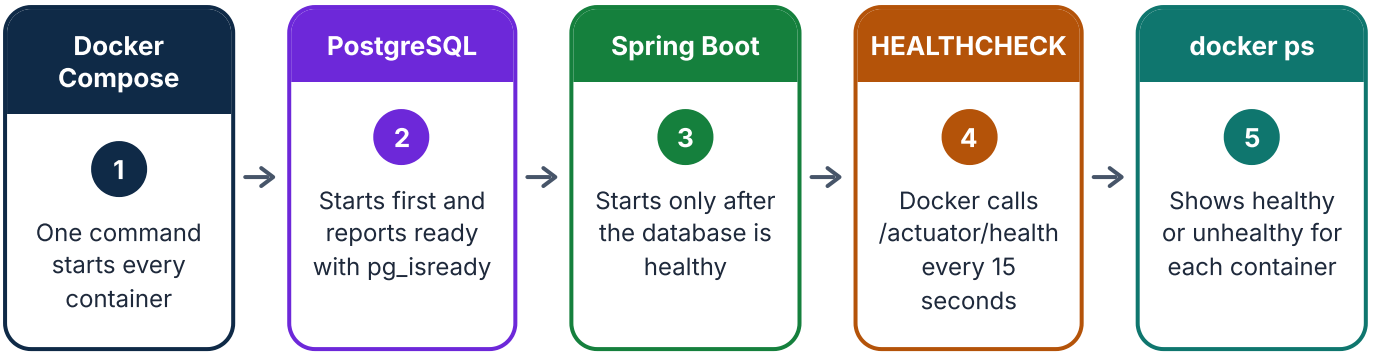
Check the database too
Health includes the PostgreSQL connection.

Start in the right order
The app waits until PostgreSQL is ready.

Repeat it easily
One command starts the whole stack.

3. How does our project work?

Everything starts with one command. Five steps take us from **docker compose up** to a clear health status:

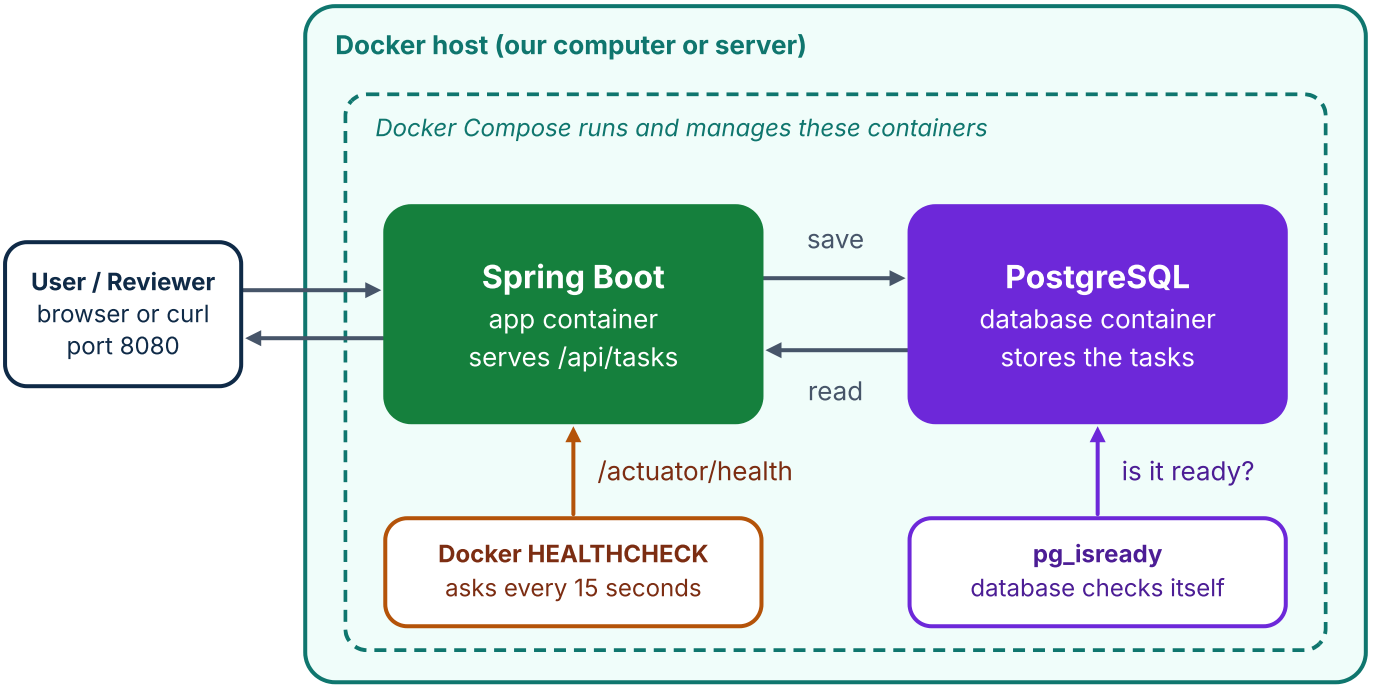


We run one command > Docker checks the app > docker ps tells the truth

Docker runs the check itself, so no extra tool or script is needed.

What happens inside Docker

Spring Boot answers requests and saves data in PostgreSQL. **Docker Compose** starts and manages both containers, and each one has its own health check.



In simple words: Docker keeps asking the app "are you OK?". If the app stops answering, or loses its database, Docker marks the container **unhealthy**.

4. What does each technology do?

Technology	Simple explanation
Spring Boot	Runs our application and its API.
Actuator	Adds the /actuator/health URL that reports UP or DOWN.
Spring Data JPA	Reads and writes our tasks in PostgreSQL.
Maven	Builds the application into one runnable JAR file.
Docker	Packages the app so it runs the same everywhere.
HEALTHCHECK	Docker's built-in test that calls the health URL again and again.
PostgreSQL	Stores our application data.
Docker Compose	Starts and manages the app and database containers together.

5. What is a HEALTHCHECK?

A HEALTHCHECK is a small test that Docker runs inside the container again and again. The result decides which of three statuses we see in **docker ps**:

starting
First 60 seconds. Failed checks do not count yet.

→

healthy
Actuator answered HTTP 200 with status UP.

→

unhealthy
3 checks in a row failed (HTTP 503 or no answer).

15s
interval
How often Docker checks

5s
timeout
Longest wait for one answer

60s
start-period
Grace time while Java boots

3
retries
Failures before "unhealthy"

Dockerfile

```
HEALTHCHECK --interval=15s --timeout=5s --start-period=60s --retries=3 \
  CMD wget -q -O /dev/null http://127.0.0.1:8080/actuator/health
```

6. What does our project prove?

When the lab runs, we can show the following:

- Docker marks the app container **healthy** when Actuator reports UP.
- The PostgreSQL container reports healthy with pg_isready.
- The app starts only after the database is ready.
- The health URL includes the database connection status.
- A simulated fault makes the app answer 503 while the container stays Up.
- docker ps turns **unhealthy**, then healthy again once the fault is cleared.

For example, the health URL answers differently before and after the fault (shortened):

HTTP 200

```
{
  "status": "UP",
  "components": {
    "db": { "status": "UP" },
    "simulatedFault": {
      "status": "UP" }
  }
}
```

HTTP 503

```
{
  "status": "DOWN",
  "components": {
    "db": { "status": "UP" },
    "simulatedFault": {
      "status": "DOWN" }
  }
}
```

```
$ docker ps  (before the fault)
healthlab_app   Up 2 minutes (healthy)
healthlab_db    Up 2 minutes (healthy)
```

```
$ docker ps  (after the fault)
healthlab_app   Up 4 minutes (unhealthy)
healthlab_db    Up 4 minutes (healthy)
```

7. What should we show our reviewers?

Follow this order. **Tip:** the app is at **http://localhost:8080**.

1. **Start the stack**
   Run docker compose up -d --build.
2. **Show the healthy status**
   Run docker ps and point to (healthy) on both containers.
3. **Show the API and the database**
   Open /actuator/health, then create and list a task at /api/tasks.
4. **Inject the fault**
   Run curl -X POST localhost:8080/lab/fault. The container still shows Up.
5. **Show the unhealthy status**
   After about a minute, docker ps shows (unhealthy) and the health URL returns 503.
6. **Recover**
   Run curl -X DELETE localhost:8080/lab/fault. The status returns to healthy.

8. Questions our reviewers may ask

Q1	**What is the main purpose of this project?** To show whether the application inside a container is really working, not only whether the container is running.
Q2	**What is a Docker HEALTHCHECK?** A test command that Docker runs inside the container at fixed intervals. Its result sets the status to healthy or unhealthy.
Q3	**Why did we use Spring Boot Actuator?** It gives us the ready-made /actuator/health URL, which reports UP or DOWN and includes the database connection.
Q4	**Why do we need PostgreSQL?** To store the application's tasks so they can be read later, and to prove the health check covers a real dependency.
Q5	**What happens if the database goes down?** Actuator reports DOWN and answers 503. After 3 failed checks, Docker marks the app container unhealthy.
Q6	**Why does the health check wait 60 seconds before it counts?** Java needs time to start and connect to the database. The start-period stops normal startup from looking like a failure.
Q7	**Does Docker restart an unhealthy container automatically?** No. Plain Docker only reports the status. Restarting needs an orchestrator or an extra tool; this lab focuses on detecting the problem.
Q8	**What is the main benefit of this project?** Problems become visible within about a minute, and one command starts the whole system in the right order.

9. A short explanation the whole team can memorize

Our project is a **Docker HEALTHCHECK for a Spring Boot application with PostgreSQL**.

The main purpose is to **know whether the application is really working**, not only whether its container is running. Spring Boot **Actuator** provides a health URL, and **Docker** calls it every 15 seconds.

If the app or its database fails three checks in a row, **docker ps shows unhealthy**. **Docker Compose** starts PostgreSQL first and the app after it. We can also simulate a fault and show the status change live.

The main benefit is that **problems become visible quickly** and the whole system starts in a **repeatable** way.

How to prepare as a team

Everyone should first understand the **overall workflow** (page 2). Then divide the presentation:

Person 1	Person 2	Person 3
Architecture	**Code + Docker files**	**Live demo**
Explain the big picture, the five steps and the three health states.	Show the Dockerfile HEALTHCHECK, docker-compose.yml and the Actuator settings.	Run docker ps, inject the fault, show unhealthy, then recover.

Quick recap

Application	Spring Boot 3 (Java 21) with a tasks API
Health URL	/actuator/health on port 8080
Packaged with	Docker (multi-stage Dockerfile)
Health check	Every 15s, unhealthy after 3 failures
Data stored in	PostgreSQL 16 (checked with pg_isready)
Started with	docker compose up -d --build
Status shown by	docker ps